

Pumas Sangrientos

Team Reference Document

Índice

1 Entorno y Configuración (macOS / Linux)	1
1.1 Instalación de Utilidades GNU en Mac	1
1.2 Alias de Compilación (~/.zshrc)	1
1.3 Script de Verificación de Tipografía (hash.sh)	1
1.4 Configuración Básica de Editor (~/.vimrc)	1
1.5 Remapear Caps Lock de Forma Nativa en macOS	1
1.6 Validar Binario Activo	1
2 Teoría de Juegos Imparciales y Teorema de Sprague-Grundy	1
2.1 Conceptos Fundamentales de Posiciones	1
2.2 El Teorema de Sprague-Grundy y la función mex	2
2.3 Implementación Eficiente de Grundy Values con DP	2
3 Basics	2
3.1 Standard Competitive Programming Template	2
4 Data Structures	3
5 Graphs	3
6 Math	3
6.1 Algebra	3
6.1.1 Hill Climbing Optimization 2D	3
6.1.2 Numerical Integration (Simpson's 1/3 Rule)	3
6.1.3 Polinomio	3
6.2 Combinatoria	4
6.2.1 Binomial Coefficients Modulo Prime	4
6.3 Geometry	4
6.3.1 Angle Radians	4
6.3.2 Angle Between Segments (Complex Coordinates)	4
6.3.3 Geometry Primitives (Complex Coordinates)	5
6.3.4 Convex Hull (Monotone Chain - Compact)	5
6.3.5 Lattice Points (Pick's Theorem)	6
6.3.6 Minimum Enclosing Circle (Welzl's Algorithm)	6
6.3.7 Integer Point	7
6.3.8 Point in Polygon (Ray Casting)	7
6.3.9 Point Location vs Line	8
6.3.10 Polygon Area (Shoelace Formula)	8
6.3.11 Segment Intersection (Anti-Overflow Version)	9
6.4 Teoria Juegos	9
6.5 Teoria Numeros	9

6.5.1 BigInt Operations (Unsigned)	9
6.5.2 Prime Sieve	10
7 Strings	11

1. Entorno y Configuración (macOS / Linux)

1.1. Instalación de Utilidades GNU en Mac

Para utilizar la suite completa de GNU (como la cabecera universal `<bits/stdc++.h>`) e igualar con precisión el entorno de evaluación del concurso, instala la última versión de GCC y las utilidades de núcleo de GNU mediante Homebrew:

```
|| brew install gcc coreutils
```

1.2. Alias de Compilación (~/.zshrc)

Agrega este alias al final de tu archivo de configuración de la terminal para compilar localmente de forma rápida.

Nota Crítica de Arquitectura (Apple Silicon): En procesadores M1/M2/M3 bajo macOS, la suite `libsanitizer` dentro de GCC para ARM64 no cuenta con soporte nativo completo en Homebrew, lo que arroja el error `ld: library 'asan' not found`. Se han removido las banderas `-fsanitize=address,undefined`, pero se conservan los diagnósticos estrictos de tipos, desbordamientos lógicos y conversiones implícitas peligrosas:

```
|| # Ajustar g++-15 a la version exacta instalada en tu equipo (ej. g++-14, g++-15)
|| alias c='g++-15 -Wall -Wconversion -Wfatal-errors -g -std=c++17'
```

Actualiza tu sesión corriendo en la terminal: `source ~/.zshrc`.

También puedes simplemente agregarlo al final como: `echo .alias c='g++-15 -Wall -Wconversion -Wfatal-errors -g -std=c++17'>> ~/.zshrc`

1.3. Script de Verificación de Tipografía (hash.sh)

Crea el archivo `hash.sh` para validar la correcta transcripción de tus plantillas de código desde el cuaderno impreso. Este script compara el hash de 6 caracteres del código limpio con el hexadecimal de KACTL.

Nota sobre Portabilidad de Hashes: Es indispensable incluir la bandera `-fpreprocessed` para evitar que el compilador inyecte macros internos específicos del procesador M2 o del sistema operativo macOS, lo cual corrompería el hash portable de KACTL:

```
|| #!/bin/bash
|| g++-15 -E -dD -P -fpreprocessed - | tr -d '[:space:]' | gmd5sum | cut -c-6
```

Otorga los permisos de ejecución necesarios en tu terminal:

```
1|| chmod +x hash.sh
```

Úsalo de la siguiente forma para verificar fallos tipográficos: `./hash.sh <codigo.cpp`

1.4. Configuración Básica de Editor (~/.vimrc)

Habilita el auto-indentado limpio para código de C++ y el mapeo de escape veloz presionando las teclas `jk` o `kj` de forma sucesiva en modo inserción:

```
1|| set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul sy on
2|| im jk <esc>
3|| im kj <esc>
```

1.5. Remapear Caps Lock de Forma Nativa en macOS

Para evitar comandos de Linux incompatibles en macOS (como `xmodmap`), puedes cambiar el mapeo directamente desde la interfaz del sistema operativo:

1. Ve a **Ajustes del Sistema** → **Teclado**.
2. Haz clic en **Atajos de teclado...** y luego selecciona **Teclas de modificación**.
3. Cambia la acción de la tecla **Bloqueo de mayúsculas** (*Caps Lock*) a **Control** o **Escape** para una navegación ultra veloz en Vim.

1.6. Validar Binario Activo

Si Homebrew actualiza o instala una versión superior de GCC, identifica con exactitud el sufijo del comando y las rutas ejecutando:

```
1|| brew info gcc
2|| ls /opt/homebrew/bin/g++-*
```

2. Teoría de Juegos Imparciales y Teorema de Sprague-Grundy

2.1. Conceptos Fundamentales de Posiciones

En la convención de juego normal (el último en mover gana), definimos de forma recursiva los estados del juego:

- **Estados Terminales:** Son posiciones P (puesto que el jugador actual no puede realizar movimientos y ha perdido).
- **Posición N (Next):** Es cualquier estado desde el cual existe **al menos un movimiento** hacia una posición P . (El jugador actual se mueve a P y le hereda la muerte al rival).

- **Posición P (Previous):** Es un estado desde el cual **todos los movimientos posibles** conducen obligatoriamente a posiciones N . (Hagas lo que hagas, dejas al rival en un estado ganador).

2.2. El Teorema de Sprague-Grundy y la función mex

El teorema dicta que cualquier juego imparcial bajo juego normal es equivalente a una pila de Nim. El tamaño de dicha pila ficticia para un estado G es su valor de Grundy, denotado como $g(G)$, y se calcula mediante la función del mínimo excluido (mex):

$$g(G) = \text{mex}(\{g(V) : G \rightarrow V\})$$

Donde $G \rightarrow V$ representa un movimiento válido desde el estado G hacia el estado V . El mex de un conjunto de enteros no negativos es el entero no negativo más pequeño que **no** pertenece a dicho conjunto (ej. $\text{mex}(\{0, 1, 3\}) = 2$, $\text{mex}(\{1, 2\}) = 0$).

2.3. Implementación Eficiente de Grundy Values con DP

Para calcular los valores de Grundy en juegos abstractos (como juegos de sustracción de fichas o movimientos en grafos acíclicos dirigidos - DAGs), mapeamos los estados usando Programación Dinámica y calculamos el mex optimizado con vectores booleanos locales o conjuntos:

```
1|| // Calcula el Grundy Value para un conjunto de estados sucesores directos
2|| int getMex(const vector<int>& transitions) {
3||     vector<bool> present(transitions.size() + 1, false);
4||     for (int val : transitions) {
5||         if (val < (int)present.size()) {
6||             present[val] = true;
7||         }
8||     }
9||     int mex = 0;
10||    while (present[mex]) mex++;
11||    return mex;
12|| }
13||
14|| // Ejemplo de DP para un juego de sustraccion de fichas
15|| // Se pueden retirar {1, 3, 4} fichas por turno.
16|| int computeGrundy(int n) {
17||     vector<int> memo(n + 1, 0);
18||     vector<int> moves = {1, 3, 4}; // Movimientos permitidos
19||
20||     for (int i = 1; i <= n; ++i) {
21||         vector<int> next_states;
22||         for (int m : moves) {
23||             if (i - m >= 0) {
24||                 next_states.push_back(memo[i - m]);
25||             }
26||         }
27||         memo[i] = getMex(next_states);
28||     }
29||     return memo[n];
30|| }
```

Estrategia ICPC: Si el valor devuelto por `computeGrundy(N)` es 0, el estado actual es una P -position (pierdes). Si es > 0 , es una N -position (ganas). Si el juego

consta de múltiples sub-juegos independientes concurrentes, la respuesta global es el XOR de los valores individuales de Grundy de cada sub-juego: $G_{total} = g(G_1) \oplus g(G_2) \oplus \dots \oplus g(G_k)$.

3. Basics

3.1. Standard Competitive Programming Template

Title: Standard Competitive Programming Template

Description: Plantilla base con optimización de I/O, macros comunes y typedefs esenciales para reducir tiempo de codificación.

Author: Pumas Sangrientos

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // === Typedefs ===
6 typedef long long ll;
7 typedef long double ld;
8 typedef pair<int, int> pii;
9 typedef pair<ll, ll> pll;
10 typedef vector<int> vi;
11 typedef vector<ll> vll;
12
13 // === Macros ===
14 #define endl '\n'
15 #define pb push_back
16 #define all(x) (x).begin(), (x).end()
17 #define sz(x) (int)(x).size()
18 #define FOR(i, a, b) for(int i = (a); i < (b); ++i)
19 #define ROF(i, a, b) for(int i = (a); i >= (b); --i)
20
21 void fast_io() {
22     ios_base::sync_with_stdio(false);
23     cin.tie(NULL);
24     cout.tie(NULL);
25 }
26
27 void solve() {
28     // Escribe tu lógica aquí
29 }
30
31 int main() {
32     fast_io();
33
34     int t = 1;
35     // cin >> t; // Descomentar si el problema tiene múltiples casos de prueba
36     while(t--) {
37         solve();
38     }
39
40     return 0;
41 }
42 }
```

4. Data Structures

5. Graphs

6. Math

6.1. Algebra

6.1.1. Hill Climbing Optimization 2D

Title: Hill Climbing Optimization 2D

Author: KACTL / botellot / ICPC BRO

Description: Heuristic search to minimize a 2D continuous function $f(x, y)$. It explores a 3x3 local grid around the current state with an exponentially decaying jump size.

Complexity: $O(\log(\text{scale}) \cdot 100 \cdot 9) \approx O(1)$ with a small constant.

Limitations:

- Strictly requires the function to be unimodal (or locally well-behaved near 'start').
- Can get trapped in local minima if the surface has multiple valleys.

Usage:

auto [val, pos] = hillClimb({0, 0}, [](P p) { return p[0]*p[0] + p[1]*p[1]; });

```
1 // Macro utilitaria de KACTL
2 template<class F>
3 pair<double, P> hillClimb(P start, F f) {
4     pair<double, P> cur(f(start), start);
5     for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
6         rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
7             P p = cur.second;
8             p[0] += dx * jmp;
9             p[1] += dy * jmp;
10            cur = min(cur, make_pair(f(p), p));
11        }
12    }
13    return cur;
14 }
15
16 cout << "Ejecutando Hill Climbing desde el origen {0.0, 0.0}..." << "\n";
17 // Le pasamos un punto inicial cualquiera, por ejemplo {0, 0}
18 auto [ans_val, ans_pos] = hillClimb({0.0, 0.0}, f);
19 cout << "--- Resultados Optimizados ---" << "\n";
20 cout << "Valor minimo encontrado: " << ans_val << " (Esperado: 10.0)" << "\n";
21 cout << "Coordenada X optima: " << ans_pos[0] << " (Esperado: 5.5)" << "\n";
22 cout << "Coordenada Y optima: " << ans_pos[1] << " (Esperado: -3.2)" << "\n";
23 return 0;
24 }
```

6.1.2. Numerical Integration (Simpson's 1/3 Rule)

Title: Numerical Integration (Simpson's 1/3 Rule)

Author: KACTL / botellot / ICPC BRO

Description: Approximates the definite integral $\int_a^b f(x) dx$ (area under the curve) using Simpson's 1/3 rule by fitting parabolas over $2n$ subintervals. Complexity: $O(N)$ where N is the number of intervals.

Limitations:

- The function 'f' must be continuous and smooth within $[a, b]$.
- If the function has sharp spikes, discontinuities, or vertical asymptotes, standard Simpson's rule will lose stability. In those cases, use Adaptive Integration instead.

Usage:

```
double area = quad(0, M_PI, [](double x) { return sin(x); }); // Returns 2.0
```

```
1 // Algoritmo de Integracion de KACTL
2 template<class F>
3 double quad(double a, double b, F f, const int n = 1000) { //
4     double h = (b - a) / 2 / n, v = f(a) + f(b); //
5     rep(i, 1, n * 2) //
6         v += f(a + i * h) * (i & 1 ? 4 : 2); //
7     return v * h / 3; //
8 }
9 // --- CÓDIGO DE PRUEBA PARA TU TERMINAL EN MAC M2 ---
10 ;
11 cout << "--- Test 1: Funcion Polinomial ---" << "\n";
12 cout << "Area de 3x^2 de 0 a 2: " << quad(0, 2, f1) << " (Esperado: 8.0)" <<
13     "\n\n";
14 // Prueba 2: Una curva trigonometrica clasica
15 // Integral de f(x) = sin(x) desde x=0 hasta x=PI
16 // El area exacta bajo una montaña de seno es exactamente 2.0
17 auto f2 = [](double x) { return sin(x); };
18 cout << "--- Test 2: Funcion Trigonometrica ---" << "\n";
19 cout << "Area de sin(x) de 0 a PI: " << quad(0, M_PI, f2) << " (Esperado:
20     2.0)" << "\n";
21 return 0;
22 }
```

6.1.3. Polinomio

Title: Polinomio

Author: KACTL / botellot / ICPC BRO

Description: Structure to store and manipulate real-coefficient polynomials. Provides linear-time evaluation using Horner's Method, analytical differentiation, and in-place synthetic division (Ruffini's Rule) by a known root.

Complexity:

- Evaluation: $O(N)$
- Differentiation: $O(N)$
- Root Division: $O(N)$

Limitations:

- Uses 'double', so it is susceptible to floating-point precision loss and catastrophic cancellation in large degrees.
- 'divroot' assumes x_0 is an exact or highly precise root; otherwise, numerical errors accumulate in the lower coefficients.

Usage:

```
Poly p{{2, -3, 1}}; // Represents  $x^2 - 3x + 2$ 
```

```
double val = p(2); // Evaluates  $P(2) \rightarrow 0$ 
```

```
p.diff(); //  $P(x)$  becomes  $2x - 3$ 
```

```
1 // Macro utilitaria para emular el estilo scannable de KACTL
2 struct Poly {
3     vector<double> a;
4     // Evalúa el polinomio en el punto x usando el Método de Horner
5     // Evita calcular potencias explícitas  $x^n$ , reduciendo el error numérico.
6     double operator() (double x) const {
7         double val = 0;
8         for (int i = sz(a); i--;) {
9             val = val * x + a[i];
10        }
11        return val;
12    }
13    // Calcula la derivada analítica del polinomio in-place
14    // Regla de la potencia:  $d/dx(c * x^i) = i * c * x^{(i-1)}$ 
15    void diff() {
16        for (int i = 1; i < sz(a); ++i) {
17            a[i - 1] = i * a[i];
18        }
19        if (!a.empty()) {
20            a.pop_back(); // El grado máximo disminuye en 1
21        }
22    }
23    // Divide el polinomio entre  $(x - x_0)$  in-place usando la Regla de Ruffini
24    // Útil para "eliminar" una raíz ya encontrada y buscar las demás en un
25    // grado menor.
26    void divroot(double x0) {
27        double b = 0;
28        for (int i = sz(a); i--;) {
29            double c = a[i];
30            a[i] = b; // Guarda el coeficiente del nuevo cociente
31            b = c + b * x0; // Calcula el residuo acumulado
32        }
33        if (!a.empty()) {
34            a.pop_back(); // Elimina el último elemento sobrante (grado se
35                           // reduce en 1)
36        }
37    }
38 };
```

6.2. Combinatoria

6.2.1. Binomial Coefficients Módulo Prime

Description: Calcula coeficientes binomiales (nCr) usando factoriales precomputados e inversos modulares. Requiere que m sea primo.

Author: botellot

Title: Binomial Coefficients Modulo Prime

```
1 const int MAXN = 1000005; // Ajustar según el problema
2 // 0 998244353
3 ll factorial[MAXN];
4 ll inv_factorial[MAXN];
5 // Exponenciación binaria para el inverso modular  $O(\log m)$ 
6 ll power(ll base, ll exp) {
7     ll res = 1;
8     base %= m;
9     while (exp > 0) {
10         if (exp % 2 == 1) res = (res * base) % m;
11         base = (base * base) % m;
12         exp /= 2;
13     }
14     return res;
```

```

15 }
16 ll modInverse(ll n) {
17     return power(n, m - 2);
18 }
19 // Precomputo en O(MAXN)
20 void precompute() {
21     factorial[0] = 1;
22     for (int i = 1; i < MAXN; i++)
23         factorial[i] = (factorial[i - 1] * i) % m;
24     // Inverso del factorial final usando Fermat
25     inv_factorial[MAXN - 1] = modInverse(factorial[MAXN - 1]);
26     // Inversos de los demás factoriales en O(MAXN)
27     for (int i = MAXN - 2; i >= 0; i--)
28         inv_factorial[i] = (inv_factorial[i + 1] * (i + 1)) % m;
29 }
30 // Cálculo en O(1)
31 ll nCr(int n, int k) {
32     if (k < 0 || k > n) return 0;
33     return factorial[n] * inv_factorial[k] % m * inv_factorial[n - k] % m;
34 }

```

6.3. Geometry

6.3.1. Angle Radians

Title: Angle <-> radians

```

1 const double PI = acos(-1.0);
2 // Convierte radianes a grados
3 double rad_to_deg(double rad) {
4     return rad * 180.0 / PI;
5 }
6 // Convierte grados a radianes (por si la necesitas al revés)
7 double deg_to_rad(double deg) {
8     return deg * PI / 180.0;
9 }

```

6.3.2. Angle Between Segments (Complex Coordinates)

Title: Angle Between Segments (Complex Coordinates)

Author: CP-Algorithms / ICPC BRO

Description: Computes the directed angle from segment AB to segment CD.

Returns the result in both radians and degrees.

Complexity: $O(1)$.

```

1 using ftype = double;
2 using point = complex<ftype>;
3 const double PI = acos(-1.0);
4 // Retorna el ángulo dirigido desde v1 hacia v2 en radianes (rango de -PI a PI)
5 // Un valor positivo significa giro antihorario, negativo significa giro horario
6 double angle_between_vectors(point v1, point v2) {
7     // La magia de los complejos: arg(conj(v1) * v2)
8     // es equivalente a hacer atan2(cross(v1,v2), dot(v1,v2))
9     return arg(conj(v1) * v2);
10 }
11 // Retorna el ángulo dirigido del segmento AB al segmento CD
12 double angle_between_segments(point a, point b, point c, point d) {
13     point v1 = b - a; // Vector del primer segmento

```

```

14     point v2 = d - c; // Vector del segundo segmento
15     return angle_between_vectors(v1, v2);
16 }
17 void solve() {
18     // Segmento 1: De (0,0) a (2,0) -> Vector horizontal a la derecha (0 grados)
19     point a = {0.0, 0.0}, b = {2.0, 0.0};
20     // Segmento 2: De (0,0) a (0,2) -> Vector vertical hacia arriba (90 grados)
21     point c = {0.0, 0.0}, d = {0.0, 2.0};
22     double rad = angle_between_segments(a, b, c, d);
23     double deg = rad * 180.0 / PI;
24     cout << "Angulo en Radianes: " << rad << "\n"; // Debería dar PI / 2
25     // (~1.5708)
26     cout << "Angulo en Grados: " << deg << "\n"; // Debería dar 90.0

```

6.3.3. Geometry Primitives (Complex Coordinates)

Title: Geometry Primitives (Complex Coordinates)

Author: CP-Algorithms / ICPC BRO

Description: 2D Geometry primitives using std::complex.

Maps (X, Y) to the complex number $X + iY$.

Provides native vector operations, dot product, cross product, and rotations.

Complexity: $O(1)$ for all primitive operations.

```

1 // Podemos usar double o int64_t dependiendo de si el problema requiere
2 // precision exacta
3 using ftype = double;
4 using point = complex<ftype>;
5 // --- OPERACIONES BASICAS NATIVAS ---
6 // punto p = {x, y};
7 // p.real() -> Accede a la coordenada X
8 // p.imag() -> Accede a la coordenada Y
9 // p1 + p2 -> Suma de vectores (Traslacion)
10 // p1 - p2 -> Resta de vectores (Vector de p2 apuntando a p1)
11 // p * k -> Multiplicacion por un escalar k (Escalamiento)
12 // Producto Punto (Dot Product): a . b = |a||b|cos(theta)
13 // Utilidad: Proyecciones, detectar angulos agudos/rectos/obtusos
14 ftype dot(point a, point b) {
15     return (conj(a) * b).real();
16 }
17 // Producto Cruz (Cross Product): a x b = |a||b|sin(theta)
18 // Utilidad: Areas, orientacion (CCW), giros izquierda/derecha
19 ftype cross(point a, point b) {
20     return (conj(a) * b).imag();
21 }
22 // Magnitud al cuadrado (Squared Magnitude)
23 // Si ftype es entero, esto se mantiene 100% exacto sin floats
24 ftype norm_sq(point a) {
25     return norm(a); // std::norm regresa x^2 + y^2
26 }
27 // Magnitud real (Distancia al origen)
28 double magnitude(point a) {
29     return abs(a); // std::abs regresa sqrt(x^2 + y^2)
30 }
31 // Distancia entre dos puntos
32 double dist(point a, point b) {
33     return abs(a - b);
34 }
35 // --- ORIENTACION Y GIROS ---
36 // Orientacion de 3 puntos (CCW)
37 // Retorna: > 0 si c gira a la izquierda de ab, < 0 a la derecha, 0 si son
38 // colineales

```

```

37 ftype ccw(point a, point b, point c) {
38     return cross(b - a, c - a);
39 }
40 // --- ROTACIONES (Solo si ftype es punto flotante) ---
41 // Angulo polar del vector en radianes (rango de -pi a pi)
42 double angle(point a) {
43     return arg(a);
44 }
45 // Rotar un punto 'a' un angulo 'theta' (en radianes) respecto al origen
46 point rotate_origin(point a, double theta) {
47     return a * polar(1.0, theta);
48 }
49 // Rotar un punto 'p' respecto a un centro 'pivot' un angulo 'theta'
50 point rotate_pivot(point p, point pivot, double theta) {
51     return pivot + (p - pivot) * polar(1.0, theta);
52 }
53 void solve() {
54     point a = {3.0, 4.0};
55     point b = {1.0, 2.0};
56     cout << "X: " << a.real() << ", Y: " << a.imag() << "\n";
57     cout << "Suma: " << (a + b).real() << " " << (a + b).imag() << "\n";
58     cout << "Producto Punto: " << dot(a, b) << "\n";
59     cout << "Producto Cruz: " << cross(a, b) << "\n";
60 }

```

6.3.4. Convex Hull (Monotone Chain - Compact)

Title: Convex Hull (Monotone Chain - Compact)

Author: botellot / ICPC BRO

Description: Computes the Convex Hull of a set of points using Andrew's Monotone Chain.

Extremely compact version using a double-pass loop and coordinate reversing.

Complexity: $O(N \log N)$ due to sorting.

Usage:

vector<P> pts = {{0,0}, {3,0}, {3,3}, {0,3}, {1,1}};

vector<P> hull = get_convex_hull(pts);

```

1 struct P {
2     ll x, y;
3     void read() {
4         cin >> x >> y; }
5     P operator - (const P& b) const { return {x - b.x, y - b.y}; }
6     ll operator * (const P& b) const { return x * b.y - y * b.x; }
7     __int128 triangle(const P& b, const P& c) const {return (__int128)((b - *
8         this) * (c - *this));}
9     bool operator < (const P& b) const {return make_pair(x,y) < make_pair(b.x, b
10         .y);}
11 };
12 // Funcion que recibe el set de puntos y retorna las esquinas del Convex Hull
13 vector<P> get_convex_hull(vector<P> points) {
14     int n = points.size();
15     if (n <= 1) return points;
16     // 1. Ordenar de izquierda a derecha
17     sort(points.begin(), points.end());
18     vector<P> hull;
19     // 2. Doble pasada (rep=0 para el cascaron superior, rep=1 para el inferior)
20     for (int rep = 0; rep < 2; ++rep) {
21         const int S = hull.size(); // S guarda el tamaño inicial de esta pasada
22         for (P C : points) {
23             // hull.size() - S >= 2 asegura no borrar puntos de la pasada
24             // anterior

```

```

25         while ((int)hull.size() - S >= 2) {
26             P A = hull[hull.size() - 2]; // Equivalente a hull.end()[-2]
27             P B = hull[hull.size() - 1]; // Equivalente a hull.end()[-1]
28             // Si el giro es a la derecha (o colineal), el punto B es valido
29             if (A.triangle(B, C) <= 0) {
30                 break;
31             }
32             // Si el giro es a la izquierda, B hace una "panza" invalida y
33             // se elimina
34             hull.pop_back();
35         }
36         hull.push_back(C);
37     }
38     hull.pop_back(); // Elimina el punto extremo duplicado antes de dar la
39     // vuelta
40     reverse(points.begin(), points.end()); // Voltrear los puntos para el
41     // viaje de regreso
42     return hull;
43 }

```

6.3.5. Lattice Points (Pick's Theorem)

Title: Lattice Points (Pick's Theorem)

Author: botellot / ICPC BRO

Description: Computes the number of interior (I) and boundary (B) lattice points of a simple polygon with integer coordinates using Pick's Theorem ($A = I + B/2 - 1$).

Complexity: $O(N \log(\max_coord))$ due to GCD for each edge.

Usage:

vector<P> poly = {{0,0}, {0,3}, {4,0}};

auto [interior, boundary] = get_lattice_points(poly); // Returns {0, 12}

```

1 // Contiene std::gcd para C++17
2 struct P {
3     ll x, y;
4     void read() { cin >> x >> y; }
5     P operator - (const P& b) const { return {x - b.x, y - b.y}; }
6     // __int128 para evitar overflow en coordenadas grandes durante el producto
7     // cruz
8     __int128 operator * (const P& b) const { return (__int128)x * b.y - (
9         __int128)y * b.x; }
10 };
11 // Funcion iterativa de GCD para asegurar compatibilidad absoluta y velocidad
12 ll safe_gcd(ll a, ll b) {
13     a = abs(a); b = abs(b);
14     while (b) {
15         a %= b;
16         swap(a, b);
17     }
18     return a;
19 }
20 // Retorna un par: {puntos_interiores, puntos_en_el_borde}
21 pair<ll, ll> get_lattice_points(const vector<P>& poly) {
22     int n = poly.size();
23     __int128 double_area = 0;
24     ll boundary = 0;
25     for (int i = 0; i < n; i++) {
26         int j = (i + 1 == n) ? 0 : i + 1;
27         // 1. Acumular el doble del area (Shoelace)
28         double_area += poly[i] * poly[j];
29         // 2. Contar puntos enteros en el segmento actual usando GCD
30         P diff = poly[j] - poly[i];

```



```

29     boundary += safe_gcd(diff.x, diff.y);
30 }
31 // Asegurar que el area sea positiva
32 if (double_area < 0) double_area = -double_area;
33 // 3. Aplicar Teorema de Pick despejando los puntos interiores (I)
34 // Formula original:  $A = I + B/2 - 1 \Rightarrow 2A = 2I + B - 2 \Rightarrow 2I = 2A - B + 2$ 
35 ll interior = (ll)(double_area - boundary + 2) / 2;
36 return {interior, boundary};
37 }
38 void solve() {
39     int n;
40     if (!(cin >> n)) return;
41     vector<P> polygon(n);
42     for (int i = 0; i < n; i++) {
43         polygon[i].read();
44     }
45     pair<ll, ll> ans = get_lattice_points(polygon);
46     cout << ans.first << " " << ans.second << "\n";
47 }
48 return 0;
49 }

```

6.3.6. Minimum Enclosing Circle (Welzl's Algorithm)

Title: Minimum Enclosing Circle (Welzl's Algorithm)

Author: CP-Algorithms / ICPC BRO

Description: Computes the Minimum Enclosing Circle (MEC) in expected $O(N)$ time.

Uses std::variant and std::complex for zero-allocation performance and clean logic.

Necesita c++17 para variant, visit y chrono

Complexity: Expected $O(N)$.

Limitations: Coordinates must fit within int64_t operations inside indicator().

```

1 using ftype = int64_t;
2 using point = complex<ftype>;
3 // El circulo minimo siempre esta representado por 2 o 3 puntos en su
4   circunferencia
5 using mec = variant<
6     array<point, 2>,
7     array<point, 3>
8 >;
9 ftype indicator(mec const& C, point z) {
10     return visit([&](auto &&C) {
11         point a = C[0], b = C[1];
12         // La parte real de Multiplicar un numero complejo por el conjugado de
13           otro (conj)
14         // es exactamente igual al producto punto de ambos vectores.
15         point I0 = (b - z) * conj(a - z);
16         if constexpr (C.size() == 2) {
17             return real(I0);
18         } else {
19             point c = C[2];
20             point I2 = (a - c) * conj(b - c);
21             point I1 = I0 * I2;
22             return imag(I2) < 0 ? -imag(I1) : imag(I1);
23         }
24     }, C);
25 }
26 bool inside(mec const& C, point p) {
27     return indicator(C, p) <= 0;
28 }

```

```

27 // Inicializacion del generador con un seed basado en tiempo real para evitar
28   hacks
29 mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
30 mec enclosing_circle(vector<point> &p) {
31     int n = p.size();
32     if (n == 0) return array<point, 2>{point{0, 0}, point{0, 0}};
33     // El shuffle aleatorio es lo que garantiza el tiempo esperado  $O(N)$ 
34     shuffle(p.begin(), p.end(), gen);
35     // Inicializar de forma segura con el primer punto duplicado (Soporta  $N=1$ 
36       perfectamente)
37     mec C = array<point, 2>{p[0], p[0]};
38     for (int i = 0; i < n; i++) {
39         if (!inside(C, p[i])) {
40             C = array<point, 2>{p[i], p[0]};
41             for (int j = 0; j < i; j++) {
42                 if (!inside(C, p[j])) {
43                     C = array<point, 2>{p[i], p[j]};
44                     for (int k = 0; k < j; k++) {
45                         if (!inside(C, p[k])) {
46                             C = array<point, 3>{p[i], p[j], p[k]};
47                         }
48                     }
49                 }
50             }
51         }
52     }
53     return C;
54 }
55 // --- BONUS: Como recuperar el centro y el radio en punto flotante si el
56   problema lo pide ---
57 pair<complex<double>, double> get_circle_geometry(mec const& C) {
58     return visit([&](auto &&C) -> pair<complex<double>, double> {
59         if constexpr (C.size() == 2) {
60             complex<double> a(real(C[0]), imag(C[0]));
61             complex<double> b(real(C[1]), imag(C[1]));
62             complex<double> center = (a + b) / 2.0;
63             double radius = abs(a - b) / 2.0;
64             return {center, radius};
65         } else {
66             complex<double> a(real(C[0]), imag(C[0]));
67             complex<double> b(real(C[1]), imag(C[1]));
68             complex<double> c(real(C[2]), imag(C[2]));
69             // Formula del circuncentro usando numeros complejos
70             complex<double> d = (a - b) * conj(c - b);
71             complex<double> center = b + (a - b) * complex<double>(0, 1) * (c -
72               b) * conj(c - a) / (2.0 * d.imag());
73             double radius = abs(a - center);
74             return {center, radius};
75         }
76     }, C);
77 }
78 void solve() {
79     int n;
80     if (!(cin >> n)) return;
81     vector<point> orig_points(n);
82     for (int i = 0; i < n; i++) {
83         ftype x, y;
84         cin >> x >> y;
85         orig_points[i] = point{x, y};
86     }
87     // Sacamos una copia para que Welzl la barajee sin perder el orden original
88     vector<point> working_points = orig_points;
89     mec C = enclosing_circle(working_points);
90 }

```

6.3.7. Integer Point

Title: Integer Point

Author: botellot / ICPC BRO

Description: Robust 2D Point/Vector structure using long long to avoid precision issues.

Includes dot product, cross product, sorting comparators.

Complexity: $O(1)$ for all operations.

Limitations:

- Does not support non-integer operations (like division or precise rotation).

- Watch out for overflow in cross/dot products if coordinates are $> 10^9$ (results fit in `__int128` if needed).

Usage:

P {a[1, 2], b[3, 4], c[5, 6];

```
1 struct P {
2     ll x, y;
3     void read() {
4         cin >> x >> y;
5     }
6     // Operaciones básicas de vectores
7     P operator + (const P& b) const { return P{x + b.x, y + b.y}; }
8     P operator - (const P& b) const { return P{x - b.x, y - b.y}; }
9     void operator += (const P& b) { x += b.x; y += b.y; }
10    void operator -= (const P& b) { x -= b.x; y -= b.y; }
11    // Multiplicación por un escalar
12    P operator * (ll k) const { return P{x * k, y * k}; }
13    // Producto Punto (Dot Product) -> a . b = |a||b|cos(theta)
14    // Utilidad: Proyecciones, ver si un ángulo es agudo (>0), recto (=0) u
15    // obtuso (<0)
16    ll dot(const P& b) const { return x * b.x + y * b.y; }
17    // Producto Cruz (Cross Product) -> a x b = |a||b|sin(theta)
18    // Utilidad: Áreas, ver orientaciones y giros
19    ll operator * (const P& b) const { return x * b.y - y * b.x; }
20    // Magnitud al cuadrado (evita usar sqrt y floats)
21    ll norm() const { return x * x + y * y; }
22    // Comparadores (Ordena puntos y puede usar std::sort)
23    // Ordena de izquierda a derecha, y abajo hacia arriba
24    bool operator < (const P& b) const {
25        return make_pair(x,y) < make_pair(b.x, b.y);
26    }
27    bool operator == (const P& b) const {
28        return x == b.x && y == b.y;
29    }
30};
```

6.3.8. Point in Polygon (Ray Casting)

Title: Point in Polygon (Ray Casting)

Author: botellot / ICPC BRO

Description: Checks if a point P is INSIDE (1), OUTSIDE (0), or on the BOUNDARY (2) of a simple polygon.

Uses a ray to infinity shifted by X+1 to completely avoid vertex-intersection edge cases.

Needs vertices of polygon ordered clockwise or anticlockwise.

Complexity: $O(N)$ per query, where N is the number of polygon vertices.

```
1 struct P {
2     ll x, y;
```

```
3     void read() { cin >> x >> y; }
4     P operator - (const P& b) const { return {x - b.x, y - b.y}; }
5     __int128 operator * (const P& b) const { return (__int128)x * b.y - (
6         __int128)y * b.x; }
7 };
8 // Determina si el punto c esta sobre el segmento ab
9 bool in_segment(P a, P b, P c) {
10    if ((a - c) * (b - c) != 0) return false; // No son colineales
11    return min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
12        min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
13 }
14 // Interseccion de segmentos
15 bool intersect(P p1, P p2, P p3, P p4) {
16    if ((p4 - p3) * (p2 - p1) == 0) {
17        if ((p2 - p1) * (p3 - p1) != 0) return false;
18        for (int rep = 0; rep < 2; rep++) {
19            if (max(p1.x, p2.x) < min(p3.x, p4.x) || max(p1.y, p2.y) < min(p3.y,
20                p4.y)) return false;
21            swap(p1, p3); swap(p2, p4);
22        }
23        return true;
24    }
25    for (int rep = 0; rep < 2; rep++) {
26        __int128 s1 = (p2 - p1) * (p3 - p1);
27        __int128 s2 = (p2 - p1) * (p4 - p1);
28        if ((s1 < 0 && s2 < 0) || (s1 > 0 && s2 > 0)) return false;
29        swap(p1, p3); swap(p2, p4);
30    }
31    return true;
32 }
33 // Retorna: 0 = OUTSIDE, 1 = INSIDE, 2 = BOUNDARY
34 int point_in_polygon(const vector<P>& poly, P p) {
35    int n = poly.size();
36    int cnt = 0;
37    // Rayo al "infinito". El truco de p.x + 1 evita que el rayo cruce
38    // exactamente
39    // por un vertice entero del poligono, eliminando bugs de doble conteo.
40    P out = {p.x + 1, 2000000007LL};
41    for (int i = 0; i < n; i++) {
42        int j = (i + 1 == n) ? 0 : i + 1;
43        // 1. Si el punto esta sobre el borde de este segmento, terminar de
44        // inmediato
45        if (in_segment(poly[i], poly[j], p)) {
46            return 2;
47        }
48        // 2. Contar intersecciones con el rayo
49        if (intersect(p, out, poly[i], poly[j])) {
50            cnt++;
51        }
52    }
53    // Impar = INSIDE, Par = OUTSIDE
54    return (cnt % 2 == 1) ? 1 : 0;
55 }
56 void solve() {
57    int n, m;
58    if (!(cin >> n >> m)) return;
59    vector<P> polygon(n);
60    for (int i = 0; i < n; i++) polygon[i].read();
61    while (m--) {
62        P p; p.read();
63        int res = point_in_polygon(polygon, p);
64        if (res == 2) cout << "BOUNDARY\n";
65        else if (res == 1) cout << "INSIDE\n";
66        else cout << "OUTSIDE\n";
67    }
68 }
```



```
64 | }
```

6.3.9. Point Location vs Line

Title: Point Location vs Line

Author: botellot / ICPC BRO

Description: Returns orientation of P3 relative to vector P1->P2.

- LEFT (1): Giro Antihorario (Counter-Clockwise). Producto cruz > 0.

- RIGHT (-1): Giro Horario (Clockwise). Producto cruz < 0.

- TOUCH (0): Colineales / Sobre la misma linea. Producto cruz = 0.

Complexity: O(1).

```
1 | struct P {
2 |     ll x, y;
3 |     P operator - (const P& b) const { return {x - b.x, y - b.y}; }
4 |     ll operator * (const P& b) const { return x * b.y - y * b.x; }
5 | };
6 | int point_location(P p1, P p2, P p3) {
7 |     ll cp = (p2 - p1) * (p3 - p1);
8 |     if (cp < 0) return -1; // Horario / Derecha
9 |     if (cp > 0) return 1;  // Antihorario / Izquierda
10 |    return 0;              // Colineal
11 | }
```

6.3.10. Polygon Area (Shoelace Formula)

Title: Polygon Area (Shoelace Formula)

Author: botellot / ICPC BRO

Description: Computes twice the area of a simple polygon (convex or concave).

Returns 2 * Area as long long to maintain absolute integer precision.

Vertices must be given in clockwise or counter-clockwise order.

Complexity: O(N) where N is the number of vertices.

Usage:

vector<P> poly = {{0,0}, {4,0}, {4,3}};

ll double_area = polygon_double_area(poly); // Returns 12

double real_area = double_area / 2.0; // 6.0

```
1 | struct P {
2 |     ll x, y;
3 |     void read() {cin >> x >> y;}
4 |     ll operator * (const P& b) const {return x * b.y - y * b.x;}
5 | };
6 | // Retorna 2 * Area del poligono usando el producto cruz desde el origen
7 | ll polygon_double_area(const vector<P>& poly) {
8 |     int n = (int)poly.size();
9 |     ll area = 0LL;
10 |    for (int i = 0; i < n; i++) {
11 |        int next_idx = (i + 1 == n) ? 0 : i + 1;
12 |        area += poly[i] * poly[next_idx];
13 |    }
14 |    return abs(area);
15 | }
16 | void solve() {
17 |     int n; cin >> n;
18 |     vector<P> polygon(n);
19 |     for(P& p : polygon) {
20 |         p.read();
```

```
21 |     }
22 |     ll area = polygon_double_area(polygon);
23 | }
```

6.3.11. Segment Intersection (Anti-Overflow Version)

Title: Segment Intersection (Anti-Overflow Version)

Author: botellot / ICPC BRO

Description: Robustly checks if line segments P1P2 and P3P4 intersect.

Uses __int128 in cross product to strictly avoid overflow with coordinates up to 10^9 .

Handles general intersections, T-junctions, and collinear/overlapping segments.

Complexity: O(1).

```
1 | struct P {
2 |     ll x, y;
3 |     void read() {
4 |         cin >> x >> y;
5 |     }
6 |     P operator - (const P& b) const {
7 |         return {x - b.x, y - b.y};
8 |     }
9 |     // Retorna __int128 para blindar multiplicaciones de coordenadas de hasta
10 |    // 10^9
11 |    __int128 operator * (const P& b) const {
12 |        return (__int128)x * b.y - (__int128)y * b.x;
13 |    }
14 | };
15 | bool intersect(P p1, P p2, P p3, P p4) {
16 |     // Caso 1: Los segmentos son paralelos o colineales
17 |     if ((p4 - p3) * (p2 - p1) == 0) {
18 |         // Si no son colineales (paralelos en distintas lineas), no hay
19 |         // interseccion
20 |         if ((p2 - p1) * (p3 - p1) != 0) return false;
21 |         // Si son colineales, revisamos el traslape de sus cajas contenedoras
22 |         for (int rep = 0; rep < 2; rep++) {
23 |             if (max(p1.x, p2.x) < min(p3.x, p4.x) || max(p1.y, p2.y) < min(p3.y,
24 |                 p4.y)) {
25 |                 return false;
26 |             }
27 |             swap(p1, p3); swap(p2, p4);
28 |         }
29 |         return true;
30 |     }
31 |     // Caso 2: No paralelos (Straddle Test con loop y swap)
32 |     for (int rep = 0; rep < 2; rep++) {
33 |         // s1 y s2 guardan los signos de orientacion usando __int128
34 |         __int128 s1 = (p2 - p1) * (p3 - p1);
35 |         __int128 s2 = (p2 - p1) * (p4 - p1);
36 |         // Se descarta si ambos puntos estan estrictamente del mismo lado
37 |         if ((s1 < 0 && s2 < 0) || (s1 > 0 && s2 > 0)) return false;
38 |         swap(p1, p3); swap(p2, p4);
39 |     }
40 |     return true;
41 | }
42 | void solve() {
43 |     P p1, p2, p3, p4;
44 |     p1.read(); p2.read(); p3.read(); p4.read();
45 |     if (intersect(p1, p2, p3, p4)) {
46 |         cout << "YES\n";
47 |     } else {
48 |         cout << "NO\n";
```

```

46     }
47 }
48     return 0;
49 }

```

6.4. Teoria Juegos

6.5. Teoria Numeros

6.5.1. BigInt Operations (Unsigned)

Description: Basic BigInt structure for positive integers using base 10^9 .

* * Complexity:

- Addition/Subtraction: $O(N)$ where N is number of nodes (digits/9).
- Multi-Int Multiply/Divide/Mod: $O(N)$.
- BigInt Multiply: $O(N * M)$ quadratic. Fails if total digits $> 10^5$.
- String Conversion: $O(L)$ where L is string length.

* Limitations:

- Only supports non-negative integers.
- less_lnum assumes $a \geq b$ (no negative result support).
- Division and Modulo are only by 'long long' scalars, not BigInt/BigInt.
- Multiplication is not FFT-optimized.

Provides fast I/O and linear operations for small/medium inputs.

Author: botellot / ICPC BRO

Title: BigInt Operations (Unsigned)

* Usage:

BigInt a("123456789123456789"), b(100);

BigInt c = prod_lnum_int(a, 5);

ll remainder = mod_lnum_int(a, 1e9+7);

```

1 struct BigInt {
2     static const int BASE = 1e9;
3     vector<int> nodes;
4     BigInt() {}
5     BigInt(long long v) {
6         if (v == 0) nodes.push_back(0);
7         while (v > 0) {
8             nodes.push_back(v % BASE);
9             v /= BASE;
10        }
11    }
12    BigInt(string s) {
13        for (int i = (int)s.length(); i > 0; i -= 9) {
14            if (i < 9)
15                nodes.push_back(stoi(s.substr(0, i)));
16            else
17                nodes.push_back(stoi(s.substr(i - 9, 9)));
18        }
19        trim();
20    }
21    void trim() {
22        while (nodes.size() > 1 && nodes.back() == 0)
23            nodes.pop_back();
24        if (nodes.empty()) nodes.push_back(0);
25    }
26 };
27 // --- OPERACIONES EXTERNAS ---
28 // O(N) - Suma estándar con carry

```

```

29 BigInt sum_lnum(const BigInt& a, const BigInt& b) {
30     BigInt res = a;
31     int carry = 0;
32     for (size_t i = 0; i < max(res.nodes.size(), b.nodes.size()) || carry; ++i)
33     {
34         if (i == res.nodes.size()) res.nodes.push_back(0);
35         long long cur = (long long)res.nodes[i] + carry + (i < b.nodes.size() ?
36             b.nodes[i] : 0);
37         res.nodes[i] = cur % BigInt::BASE;
38         carry = cur / BigInt::BASE;
39     }
40     return res;
41 }
42 // O(N) - Resta estándar (Asume a >= b)
43 BigInt less_lnum(const BigInt& a, const BigInt& b) {
44     BigInt res = a;
45     int carry = 0;
46     for (size_t i = 0; i < b.nodes.size() || carry; ++i) {
47         long long cur = res.nodes[i] - carry - (i < b.nodes.size() ? b.nodes[i]
48             : 0);
49         carry = cur < 0;
50         if (carry) cur += BigInt::BASE;
51         res.nodes[i] = (int)cur;
52     }
53     res.trim();
54     return res;
55 }
56 // O(N) - Multiplicación por escalar
57 BigInt prod_lnum_int(const BigInt& a, ll v) {
58     if (v == 0) return BigInt(0);
59     BigInt res = a;
60     ll carry = 0;
61     for (size_t i = 0; i < res.nodes.size() || carry; ++i) {
62         if (i == res.nodes.size()) res.nodes.push_back(0);
63         long long cur = res.nodes[i] * 1LL * v + carry;
64         res.nodes[i] = cur % BigInt::BASE;
65         carry = cur / BigInt::BASE;
66     }
67     res.trim();
68     return res;
69 }
70 // O(N*M) - Multiplicación BigInt * BigInt
71 BigInt prod_lnum(const BigInt& a, const BigInt& b) {
72     BigInt res;
73     res.nodes.resize(a.nodes.size() + b.nodes.size(), 0);
74     for (size_t i = 0; i < a.nodes.size(); ++i) {
75         ll carry = 0;
76         for (size_t j = 0; j < b.nodes.size() || carry; ++j) {
77             long long cur = res.nodes[i + j] +
78                 a.nodes[i] * 1LL * (j < b.nodes.size() ? b.nodes[j]
79                     : 0) + carry;
80             res.nodes[i + j] = cur % BigInt::BASE;
81             carry = cur / BigInt::BASE;
82         }
83     }
84     res.trim();
85     return res;
86 }
87 // O(N) - División por escalar
88 BigInt div_lnum_int(const BigInt& a, ll v) {
89     BigInt res = a;
90     ll carry = 0;
91     for (int i = (int)res.nodes.size() - 1; i >= 0; --i) {
92         long long cur = res.nodes[i] + carry * 1LL * BigInt::BASE;
93         res.nodes[i] = (int)(cur / v);
94     }

```

7. Strings

```

90     carry = cur % v;
91 }
92 res.trim();
93 return res;
94 }
95 // O(N) - Módulo por escalar
96 ll mod_lnum_int(const BigInt& a, ll v) {
97     ll m = 0;
98     for (int i = (int)a.nodes.size() - 1; i >= 0; --i)
99         m = (a.nodes[i] + m * 1LL * BigInt::BASE) % v;
100     return m;
101 }
102 // O(N) - Print formateado con ceros a la izquierda para la base
103 void print(const BigInt& a) {
104     if (a.nodes.empty()) { printf("0\n"); return; }
105     printf("%d", a.nodes.back());
106     for (int i = (int)a.nodes.size() - 2; i >= 0; --i)
107         printf("%09d", a.nodes[i]);
108     printf("\n");
109 }

```

6.5.2. Prime Sieve

Title: Prime Sieve

Description: Criba para encontrar primos menores a n y contar primos en segmento [L,R]

Author: bot

```

1 //
2 void criba_n(int n){
3     vector<bool> is_prime(n, true);
4     is_prime[0] = is_prime[1] = false;
5     for (int i = 2; i <= n; i++) {
6         if (is_prime[i] && (long long)i * i <= n) {
7             for (int j = i * i; j <= n; j += i)
8                 is_prime[j] = false;
9         }
10    }
11 }
12 // Cuenta primos en rango con L hasta 1e12 y R-L~1e7
13 vector<char> segmentedSieve(long long L, long long R) {
14     // generate all primes up to sqrt(R)
15     long long lim = sqrt(R);
16     vector<char> mark(lim + 1, false);
17     vector<long long> primes;
18     for (long long i = 2; i <= lim; ++i) {
19         if (!mark[i]) {
20             primes.emplace_back(i);
21             for (long long j = i * i; j <= lim; j += i)
22                 mark[j] = true;
23         }
24     }
25     vector<char> isPrime(R - L + 1, true);
26     for (long long i : primes)
27         for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
28             isPrime[j - L] = false;
29     if (L == 1)
30         isPrime[0] = false;
31     return isPrime;
32 }

```